

# Revisiting Multi-Object Tracking Baselines: Hyperparameter Optimization with Multi-Fidelity Greedy Coordinate Search

Momir Adžemović<sup>a</sup>

<sup>a</sup>*Department of Computer Science, Faculty of Mathematics, University of Belgrade, Studentski trg 16, Belgrade, 11000, Serbia*

---

## Abstract

Multi-object tracking (MOT) is dominated by the tracking-by-detection paradigm, whose methods typically rely on a small set of hyperparameters that are conventionally chosen by hand. Tuning them requires repeated expert-guided experimentation, while the procedures used to select reported values are often not systematically evaluated or fully documented. Hyperparameter optimization (HPO) automates this process, yet it remains rarely used in MOT, and existing studies applying HPO to MOT predate modern deep-detector-based trackers and HOTA evaluation. We systematically apply HPO across two datasets and four tracking-by-detection methods. We also propose Multi-Fidelity Greedy Coordinate Search (MFGCS), which optimizes one hyperparameter at a time by first evaluating candidate values on a small subset of scenes and re-evaluating only promising candidates on the full dataset. Across all eight tracker–dataset combinations, the Tree-structured Parzen Estimator (TPE) and MFGCS outperform both our hand-tuned configurations and the corresponding published results, with improvements of up to 4.38 and 16.05 HOTA points, respectively. MFGCS also reaches a predefined HOTA target faster than TPE in seven of the eight combinations. Within each tracker–dataset pair, all optimizers share the same search space and evaluation pipeline, isolating the effect of the search strategy. We release the code and tuned configurations to enable future work to compare against systematically optimized rather than default or manually tuned baselines.

*Keywords:* multi-object tracking, hyperparameter optimization, multi-fidelity optimization, tracking-by-detection, benchmark

---

## 1. Introduction

Multi-object tracking (MOT) aims to localize objects in a video while maintaining their identities over time. It is a core component of video-based perception in autonomous driving [17], robotics [36], sports analytics [12], surveillance [33], animal behaviour analysis [11], and retail analytics [1]. The dominant paradigm is tracking-by-detection, in which an object detector is applied independently to each frame and a separate association step links the resulting detections to existing tracks [9, 38]. Its main advantage is modularity: the detector and association pipeline can be developed and replaced independently, allowing a stronger detector to be substituted without retraining the tracker. However, the association stage remains largely based on hand-designed methods whose complexity has increased over time.

---

*Email address:* pd222011@alas.matf.bg.ac.rs (Momir Adžemović)

The pioneering SORT method combines a Kalman filter with Intersection-over-Union matching [9], while Deep SORT adds appearance embeddings to the association cost [34]. Later methods incorporate further mechanisms, including a second matching pass over low-confidence detections [38], camera-motion compensation, observation-centric motion modelling, pseudo-depth ordering, and detection-confidence modulation [4, 10, 24, 37, 3]. While these mechanisms can improve tracking performance, they also introduce additional hyperparameters that must be manually specified or tuned. Common examples include detection and association thresholds, gating margins, and track lifecycle parameters, with each additional association cue introducing further settings. As a result, even relatively simple tracking-by-detection methods can depend substantially on how their hyperparameters are selected.

In practice, selecting these hyperparameters often requires substantial manual tuning [31]. For the trackers evaluated in this study, manual tuning required several hours to approximately one day per tracker. Moreover, the benchmark results reported for the considered methods [9, 38, 24, 2] generally provide little detail about the procedure used to select their hyperparameters, making the amount of tuning effort difficult to assess. This creates two problems. First, comparisons between a new tracker and its predecessor may conflate the contribution of the proposed component with differences in tuning effort. Second, a configuration selected on one benchmark may transfer poorly to another. This issue is particularly relevant for DanceTrack [32] and SportsMOT [12], whose motion and appearance characteristics differ substantially from those of the benchmarks on which many tracking-by-detection methods were originally developed. We therefore use these datasets to study the effect of systematic retuning and find substantial performance gains from optimizing tracker hyperparameters separately for each benchmark.

These issues motivate treating tracker tuning as a hyperparameter optimization (HPO) problem, replacing the manual tuning loop with an automated search within a predefined search space and evaluation budget [6, 14]. MOT is particularly well suited to this formulation because tracker performance is effectively a black-box objective with respect to its hyperparameters, as the discrete tracking and evaluation pipeline provides no usable gradient for optimization. Evaluations are also expensive, while performance can be estimated using different amounts of data, making the evaluated data subset a natural source of fidelity. Despite this fit, HPO remains rarely used in MOT. When applied, prior work typically considers a single search strategy on a single tracking domain or benchmark [15, 26], and the one study that does compare several optimizers [27] predates deep-detector-based trackers and HOTA-based evaluation, as detailed in Section 2.

In this work, we systematically study the choice of optimization strategy for MOT hyperparameter tuning. We first compare eight optimization methods spanning six families under a common search space using SORT on DanceTrack. The two strongest methods are then evaluated across two datasets, DanceTrack and SportsMOT, and four tracking-by-detection methods: SORT [9], ByteTrack [38], SparseTrack [24], and MoveSORT-KF [2]. Each tracker has its own search space, which is shared by all optimization methods used to tune it. We also propose Multi-Fidelity Greedy Coordinate Search (MFGCS), an optimization method designed to reduce the cost of full-dataset evaluations. MFGCS tunes one hyperparameter at a time, first comparing candidate values on a small subset of scenes to obtain a cheap but noisy estimate of their relative performance. The full dataset is evaluated only for a proposed update, which is accepted only if it improves the full-dataset score.

Our contributions are as follows:

- We introduce Multi-Fidelity Greedy Coordinate Search (MFGCS), a coordinate-wise opti-

mizer that first compares candidate values using inexpensive evaluations on scene subsets and evaluates the complete dataset only for promising updates. A change is accepted only if it improves the complete-dataset objective. We further ablate its three interchangeable components.

- We provide a systematic study of hyperparameter optimization for modern tracking-by-detection MOT. We compare eight optimizers from six families under a controlled search space, and evaluate the strongest methods, MFGCS and TPE, across two datasets and four trackers. To our knowledge, this is the first broad comparison of HPO strategies for modern tracking-by-detection methods, the first using HOTA as the optimization objective, and the first to evaluate multi-fidelity HPO in this setting.
- We show that commonly used tracker configurations substantially underestimate the performance of existing baselines. MFGCS and TPE improve the manually tuned configuration in all eight tracker–dataset combinations, with gains of up to 4.38 HOTA points, and re-tuning reduces the apparent improvement of newer trackers over their predecessors. These results show that conclusions about architectural progress can be confounded by differences in hyperparameter tuning.
- We verify that these gains transfer to held-out test data: for every tracker–dataset pair, the better validation-selected configuration exceeds the corresponding published test result, with improvements of up to 8.69 HOTA points. We release the code and optimized configurations so that future work can compare against systematically tuned rather than default or manually selected baselines.

## 2. Related work

We review three lines of related work. First, we cover black-box hyperparameter optimization, which encompasses the main optimizer families evaluated in this study. Second, we discuss multi-fidelity hyperparameter optimization, the line of work most closely related to MFGCS. Finally, we review prior applications of hyperparameter optimization to multi-object tracking.

**Black-box hyperparameter optimization.** Non-model-based methods explore the search space without constructing an explicit model of the objective. Random search is a standard baseline and was empirically shown to match or outperform grid search on the search spaces studied by Bergstra and Bengio [6]. Model-based methods instead fit a surrogate to previous evaluations and use it to select promising candidates [30, 18, 14]. The Tree-structured Parzen Estimator (TPE) models the densities of well-performing and remaining configurations separately and favors candidates that are more likely under the former [7]. Gaussian-process Bayesian optimization (GP-BO) places a posterior over the objective and selects candidates by optimizing an acquisition function [30], while SMAC replaces the Gaussian process with a random forest, making it well suited to mixed and conditional search spaces [18]. In our Bayesian optimization experiments, we use expected improvement, which scores candidates according to their expected improvement over the best observed value. A separate class of derivative-free methods is direct search, including coordinate descent [35] and pattern search [21], which optimize the objective without gradients or a learned surrogate by systematically exploring local changes in the search space. We represent this family with Greedy Coordinate Search (GCS), the single-fidelity coordinate-wise baseline for MFGCS. The other black-box methods evaluated in this work are random search, TPE, and GP-BO.

**Multi-fidelity optimization.** When a cheaper approximation of the objective is available, the evaluation budget can be allocated adaptively across fidelity levels. Many configurations can first be evaluated at low fidelity, with additional resources assigned only to the more promising ones. Successive Halving follows this principle by evaluating a large set of configurations with a small resource allocation and repeatedly promoting a fraction of the best-performing configurations to higher fidelities [20]. Hyperband runs multiple Successive Halving brackets with different trade-offs between the number of configurations and the resources allocated to each [22]. BOHB replaces Hyperband’s random configuration sampling with a TPE-style model [13], while ASHA adapts Successive Halving to asynchronous parallel execution [23]. Unlike model training, tracker hyperparameter evaluation does not naturally provide epochs or training iterations as a fidelity variable. We therefore define fidelity by the amount of data evaluated, using subsets of scenes for lower-fidelity evaluations and the complete dataset for the highest fidelity. We evaluate Hyperband and BOHB using this fidelity axis. MFGCS uses the same notion of fidelity but applies it within coordinate-wise search: low-fidelity evaluations compare candidate values for a single hyperparameter around the current configuration, while the complete dataset determines whether the proposed update is accepted. In our experiments, MFGCS outperforms both standard Hyperband and BOHB under this common fidelity definition.

**Hyperparameter optimization applied to MOT.** Applications of hyperparameter optimization to MOT remain sparse. To our knowledge, the only prior study that directly compares multiple optimizers is Madrigal et al. [27], which evaluates MCMC, SMAC, TPE, and Spearmint on PETS09 and ETH. That work predates the current generation of deep-detector-based trackers and HOTA-based evaluation, and does not consider multi-fidelity or direct-search methods. More recent studies have used HOTA directly as a tuning objective on a UA-DETRAC sequence [28], in the winning entry of the Fish Tracking Challenge 2024 [19], and on a trawl-camera dataset [26], but each applies a single search strategy within a single domain. The closest prior study to ours is Lurbur et al. [26], which also tunes four trackers using HOTA, but does so with one optimizer on one domain-specific dataset. In contrast, we systematically compare multiple HPO strategies, including multi-fidelity and direct-search methods, across several modern tracking-by-detection trackers and two public datasets under a common evaluation protocol.

### 3. Tracker hyperparameter optimization

We formulate tracker tuning as a single-objective black-box optimization problem and define a common budget unit for comparing optimizers throughout the paper. A common budget is necessary because the evaluated methods can incur substantially different costs per evaluation, making the number of evaluations alone an unsuitable basis for comparison.

Let  $D = \{s_1, s_2, \dots, s_{|D|}\}$  denote a dataset split consisting of  $|D|$  scenes, and let  $\theta \in \Theta$  denote a vector of tracker hyperparameters from a bounded search space  $\Theta$ . We define  $f(\theta; D)$  as the tracking performance obtained by evaluating the tracker configured with  $\theta$  on  $D$ ; the metric used throughout this work is defined in Section 5.1. The optimization problem is

$$\theta^* = \arg \max_{\theta \in \Theta} f(\theta; D), \quad (1)$$

where  $\theta^*$  denotes the optimal configuration in the search space. The objective is treated as a black box because the discrete tracking and evaluation pipeline provides no usable gradient with respect to the tracker hyperparameters.

Optimizers that evaluate candidates on subsets of scenes incur different costs per trial, making trial counts unsuitable for comparing computational effort. We therefore use the *cumulative number of scenes evaluated* as a common cost measure throughout the paper,

$$B = \sum_{i=1}^N |D_i|, \quad (2)$$

where  $i$  indexes every objective evaluation performed during an optimization run and  $D_i \subseteq D$  is the set of scenes used in that evaluation. Thus, every evaluation contributes to  $B$ , including the initial evaluation, low-fidelity candidate evaluations, and full-dataset evaluations, regardless of whether it results in an accepted update or a completed optimizer trial. Because a common trial count does not imply equal computational cost across optimizers, we report the cumulative number of scenes actually evaluated by each run. This measure assumes that scenes within a dataset have approximately similar lengths. When scene lengths differ substantially, the cumulative number of evaluated frames would provide a more accurate measure of computational effort.

#### 4. Multi-Fidelity Greedy Coordinate Search

We designed MFGCS around two characteristics of the tracker-tuning problem described in Section 3. First, single-fidelity methods such as TPE, random search, and GP-BO evaluate every proposed candidate on the complete dataset split, including candidates that are ultimately unpromising. MFGCS instead compares candidate values on a small subset of scenes and performs a complete-dataset evaluation only for candidates that pass this initial comparison. This is intended to reach comparable tracking performance while requiring substantially fewer evaluated scenes.

The second property concerns the reliability of low-fidelity evaluations. A score measured on a small subset of scenes can be a noisy estimate of performance on the complete split, with much of the variation arising from which scenes are sampled. MFGCS therefore evaluates all candidate values for a given hyperparameter on the same scene subset. This controls for scene-selection effects and makes their relative performance more informative, even when the absolute low-fidelity scores are inaccurate. The subset evaluation is therefore used only to identify the most promising candidate; an update is accepted only after it improves the score on the complete dataset.

We further assume that the objective exhibits useful local structure along individual coordinates, such that varying one hyperparameter while holding the others fixed can reveal consistent improvements. This assumption is only approximate because tracker hyperparameters interact, but in practice the objective is sufficiently structured for coordinate-wise search to provide an effective optimization strategy.

##### 4.1. Algorithm

Algorithm 1 summarizes the procedure. The outer loop sweeps over the coordinates of the hyperparameter vector  $\theta \in \Theta$ . For each coordinate, the scene sampler selects a subset  $D_s$  of  $m$  scenes, and a coordinate optimizer searches the selected hyperparameter on  $D_s$  while holding all remaining coordinates fixed. The search is anchored at the current value, so if no candidate improves upon it on the subset, the coordinate remains unchanged and the full-dataset evaluation is skipped. Otherwise, the proposed candidate is accepted only if it improves the full-dataset score by more than  $\varepsilon$ . Thus, the subset evaluation determines which candidates are considered for an update, while the full-dataset evaluation determines whether the update is accepted.

---

**Algorithm 1:** Multi-Fidelity Greedy Coordinate Search

---

**Input:** search space  $\Theta$ , dataset  $D$ , scene sampler  $S$ , coordinate optimizer  $C$ , max sweeps  $T$ , subset size  $m$ , acceptance threshold  $\varepsilon$

**Output:** configuration  $\theta$  and its full-dataset score  $v$

```
1  $\theta \leftarrow$  initial configuration
2  $v \leftarrow f(\theta; D)$  // bootstrap full-dataset score
3 for  $t \leftarrow 1$  to  $T$  do
4    $improved \leftarrow$  FALSE
5   foreach coordinate  $p$  of  $\theta$  do
6      $D_s \leftarrow S(D, m)$  // low-fidelity scene subset
7      $\theta' \leftarrow C(\theta, p, D_s)$  // search one coordinate
8     if  $\theta'_p = \theta_p$  then
9       continue // anchored; skip the doomed full evaluation
10    end
11     $v' \leftarrow f(\theta'; D)$  // high-fidelity acceptance gate
12    if  $v' > v + \varepsilon$  then
13       $\theta, v \leftarrow \theta', v'$ 
14      shrink  $\Theta_p$  around  $\theta_p$ 
15       $improved \leftarrow$  TRUE
16    end
17  end
18  if  $\neg improved$  then break
19 end
20 return  $\theta, v$ 
```

---

#### 4.2. Components

Algorithm 1 comprises three configurable components: a scene sampler, a coordinate optimizer, and a per-coordinate early-stopping rule. We describe our choices below. Section 5.4 ablates the scene sampler and the coordinate optimizer; the early-stopping rule is held fixed at the setting given here throughout our experiments.

**Scene sampler.** The scene sampler selects a subset  $D_s \subset D$  by sampling  $m$  scenes uniformly without replacement. All candidate values for a given coordinate are evaluated on the same subset, and only the proposed value returned by the coordinate optimizer is evaluated on the full dataset. Using a common subset is important because it keeps the low-fidelity comparison between candidate values consistent.

**Coordinate optimizer.** The coordinate optimizer searches a single hyperparameter and returns a candidate for full-dataset evaluation. Continuous parameters are optimized using coarse-to-fine grid search: the optimizer evaluates  $g$  evenly spaced points on the current interval  $[A, B]$ , including both endpoints, retains the best-performing value, contracts the interval around it, and repeats the procedure for  $r$  rounds. Figure 1 illustrates one such search. Integer-valued parameters use the same procedure, with each candidate rounded to the lattice and duplicate values evaluated only once. Categorical parameters have no natural ordering suitable for interval refinement, so the optimizer enumerates all levels instead. After a numeric value  $\hat{\theta}_p$  is accepted on an interval  $[A, B]$  of width  $w$ , we narrow the interval to  $[\hat{\theta}_p - \rho w, \hat{\theta}_p + \rho w]$ , clipped to  $[A, B]$ , with integer endpoints

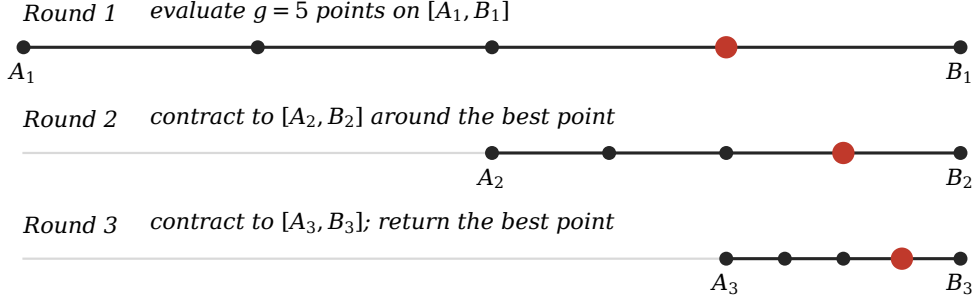


Figure 1: Coarse-to-fine grid search on one continuous coordinate ( $g = 5$ ,  $r = 3$ ). Each round keeps the best point (red) and contracts around it, giving  $[A_1, B_1] \supset [A_2, B_2] \supset [A_3, B_3]$ . The best point of the final round supplies the coordinate’s value in the candidate configuration that is then evaluated on the full dataset.

re-rounded to the lattice when necessary. Categorical search spaces are not narrowed. We compare this grid search against ternary-section and random-candidate alternatives in Section 5.4, which describes all three, and Appendix Appendix C.2 derives the trade-off between grid and ternary formally.

**Per-coordinate early stopping.** A coordinate is removed from subsequent sweeps after producing no accepted update for two consecutive sweeps, concentrating the remaining evaluations on coordinates that continue to yield improvements. The outer loop terminates when an entire sweep produces no accepted update, when the sweep limit  $T$  is reached, or when the full-dataset evaluation budget is exhausted. The bootstrap evaluation counts toward this budget.

## 5. Experimental evaluation

We evaluate hyperparameter optimization on two multi-object tracking datasets, DanceTrack [32] and SportsMOT [12], using four tracking-by-detection methods. We first describe the experimental setup, then identify the two optimization methods used in the full study, present the main results, and conclude with ablation experiments.

### 5.1. Experimental setup

**Hardware and software.** All runs used a single machine with a 12th-generation Intel Core i7-12700K CPU and an NVIDIA GeForce RTX 3070 GPU. All experiments were run with the Motrack library, which is publicly available at <https://github.com/Robotmurlock/Motrack> together with the optimized configurations of Tables B.9 and B.10. Every run uses seed 42.

**Datasets.** DanceTrack<sup>1</sup> consists of group dance recordings in which the targets share a nearly identical appearance and move non-linearly, with frequent crossings and re-entries [32]. Appearance cues are therefore largely uninformative, so the benchmark isolates the association component of a tracker. Its validation split has 25 scenes. SportsMOT<sup>2</sup> consists of basketball, football, and volleyball sequences recorded with a moving camera [12], where targets move fast and irregularly and camera motion adds apparent displacement that the motion model must absorb. Its validation

<sup>1</sup>DanceTrack GitHub page: <https://github.com/DanceTrack/DanceTrack>.

<sup>2</sup>SportsMOT GitHub page: <https://github.com/MCG-NJU/SportsMOT>.

split has 45 scenes and its test split is more than three times that size. We run every search on the validation splits, and use the test splits only for the final evaluation.

**Evaluation metrics.** We follow the TrackEval definitions [25] and report HOTA together with its detection accuracy (DetA) and association accuracy (AssA) components, as well as MOTA [8], IDF1 [29], and the number of identity switches (IDSW). We use HOTA as the optimization objective because it jointly captures detection and association performance through a geometric-mean formulation, providing a single scalar objective without introducing an explicit weighting between the two. We additionally report association throughput for each tracker in Appendix Appendix B.1, but do not include it in the optimization objective. With cached detections and the hardware described above, the association stage runs at 698–1121 frames per second.

**Optimizer metrics.** We compare optimization runs using the cumulative scene budget defined in Equation 2, which accounts for all evaluations performed and provides a hardware-independent measure of search effort. We report wall-clock time only as *time-to-target*, defined as the time at which the running-best HOTA first reaches a predefined target. This metric is reported only in Table 4, where all eight runs use the same execution configuration and are therefore directly comparable. Time-to-target measures the cost of reaching a useful configuration rather than completing the entire search, which is important because an optimizer that finds a strong configuration early can be terminated early.

**Trackers.** We tune four related tracking-by-detection methods with increasingly different association and motion mechanisms. SORT [9] provides the basic pipeline: a Kalman filter predicts track locations, detections are associated to tracks using Intersection-over-Union and the Hungarian algorithm, and lifecycle parameters control track retention and confirmation. ByteTrack [38] extends this pipeline with a second association stage over low-confidence detections. SparseTrack [24] further partitions detections by pseudo-depth and performs association stratum by stratum within this two-stage scheme. MoveSORT-KF [2] modifies both motion modelling and association by incorporating camera-motion compensation from BoT-SORT [4] and replacing the IoU cost with the Move cost. Together, these trackers form a related family of tracking-by-detection pipelines while exposing different sets of tunable association, motion, and lifecycle parameters. We restrict the study to this family and therefore do not include appearance-based or observation-centric variants such as Deep SORT [34] and OC-SORT [10].

**Object detection.** We use the public YOLOX-X [16] detectors released with each dataset. The detector is frozen and identical across every configuration, optimizer, and tracker within a dataset, so differences between the runs reported here originate in the tracker configuration. Tracker outputs are postprocessed before scoring, following the pipelines used in the tracker papers. Detector configurations and postprocessing settings are given in Appendix Appendix A.1 and Appendix Appendix A.2.

**Optimization methods.** Table 1 lists the eight methods we consider. They span six families: model-free random search, the two model-based samplers TPE and GP-BO, the two multi-fidelity schedules Hyperband and BOHB, and greedy coordinate search, of which MFGCS is the multi-fidelity version. Two of the eight are variants rather than separate families, namely TPE warm-started from a prior and coordinate search without the multi-fidelity stage. Section 5.2 selects two of the eight for the full experiment. Every method operates on the same search space for a given tracker, and on the same space for both datasets, so the comparison isolates the search strategy. Each run is capped at 100 full-fidelity trials, which is a stopping condition rather than an equal computational budget, since cost per trial differs across methods. The settings of each method and

Table 1: The eight optimization methods considered, spanning six families. Actual cost is the cumulative scene count of Equation 2, reported with the results. The two carried into the cross-tracker comparison are in bold.

| Optimizer          | Family                        | Key settings                                        |
|--------------------|-------------------------------|-----------------------------------------------------|
| Random             | Model-free                    | uniform sampling                                    |
| <b>TPE</b>         | <b>Model-based</b>            | $\gamma = 0.20$ , <b>24 EI candidates</b>           |
| TPE + prior        | Model-based                   | warm-started from the manual config                 |
| GP-BO              | Model-based                   | expected improvement (EI), 20 startup trials        |
| Random + Hyperband | Multi-fidelity                | rung schedule over scene subsets                    |
| BOHB               | Multi-fidelity                | TPE sampling with Hyperband pruning                 |
| GCS                | Direct                        | greedy coordinate search, no subset stage           |
| <b>MFGCS</b>       | <b>Multi-fidelity, direct</b> | <b>grid</b> , $g = 5$ , $r = 3$ , $m = 6$ , $T = 6$ |

the full search spaces are listed in Appendix Appendix A.4 and Appendix Appendix A.3.

### 5.2. Choosing the optimization methods

Running every optimization method on every tracker–dataset pair would already require  $8 \times 4 \times 2 = 64$  optimization runs, each taking approximately two to seven hours on the hardware described in Section 5.1. More importantly, this count assumes a single fixed configuration for each optimizer. The optimizers themselves expose hyperparameters that must be selected to ensure a fair comparison; even a modest average of 16 candidate settings per optimizer would expand the full factorial study to more than 1000 optimization runs, corresponding to months of continuous compute. We therefore adopt a two-stage evaluation protocol. First, we compare all eight optimization methods under a common search space using SORT on DanceTrack-val. We then select the two strongest methods from this controlled comparison and evaluate them across all four trackers and both datasets.

Table 2 reports the outcome. The single-fidelity samplers pay the full per-trial cost on every candidate and so consume 2500 scenes for 100 trials, whereas MFGCS reaches a higher HOTA at roughly half that budget. The two Hyperband-pruned methods are cheaper than those single-fidelity samplers, at about 1530 scenes, but score lowest of the eight; we return to them in Appendix Appendix C.3. MFGCS records the highest observed score at 53.69%. The highest-scoring sampler is TPE warm-started from the manually tuned configuration at 53.35%, which we tried out of curiosity about whether a prior helps in general, with plain TPE just behind at 53.27%. We select MFGCS and plain TPE for the broader experiment, because warm-starting would make the sampler depend on the hand-tuned configuration the benchmark exists to replace. Figure 2 shows more detailed results as running-best curves, with MFGCS in the upper-left region: it reaches a higher score than any other method at every budget beyond its first few hundred scenes.

### 5.3. Main results

Each tracker is paired with the better of TPE and MFGCS on its validation split, and the selected configuration is then evaluated unchanged on the test split and compared against the number published for that same tracker on that same split, with our metrics returned by the official evaluation servers. Table 3 reports the outcome: every published baseline is beaten, by up to 6.15 HOTA points on DanceTrack and 8.69 on SportsMOT. For SORT, tuning improves both association accuracy, from 32.60 to 36.89 AssA, and detection accuracy, from 72.00 to 79.41 DetA, lifting HOTA

Table 2: Best result per optimization method on the SORT and DanceTrack-val search space. Scenes is the cumulative count of Equation 2 and is the cost axis; trials are bookkeeping. The highest HOTA is in bold. TPE and MFGCS are the two methods carried into the cross-tracker experiment, for the reasons given in the text.

| Method             | HOTA         | Trials    | Scenes      |
|--------------------|--------------|-----------|-------------|
| Random + Hyperband | 51.46        | 100       | 1534        |
| BOHB               | 51.71        | 100       | 1530        |
| Random             | 52.28        | 100       | 2500        |
| GP-BO              | 52.50        | 100       | 2500        |
| TPE                | 53.27        | 100       | 2500        |
| TPE + prior        | 53.35        | 100       | 2500        |
| GCS                | 53.59        | 12        | 3950        |
| <b>MFGCS</b>       | <b>53.69</b> | <b>17</b> | <b>1297</b> |

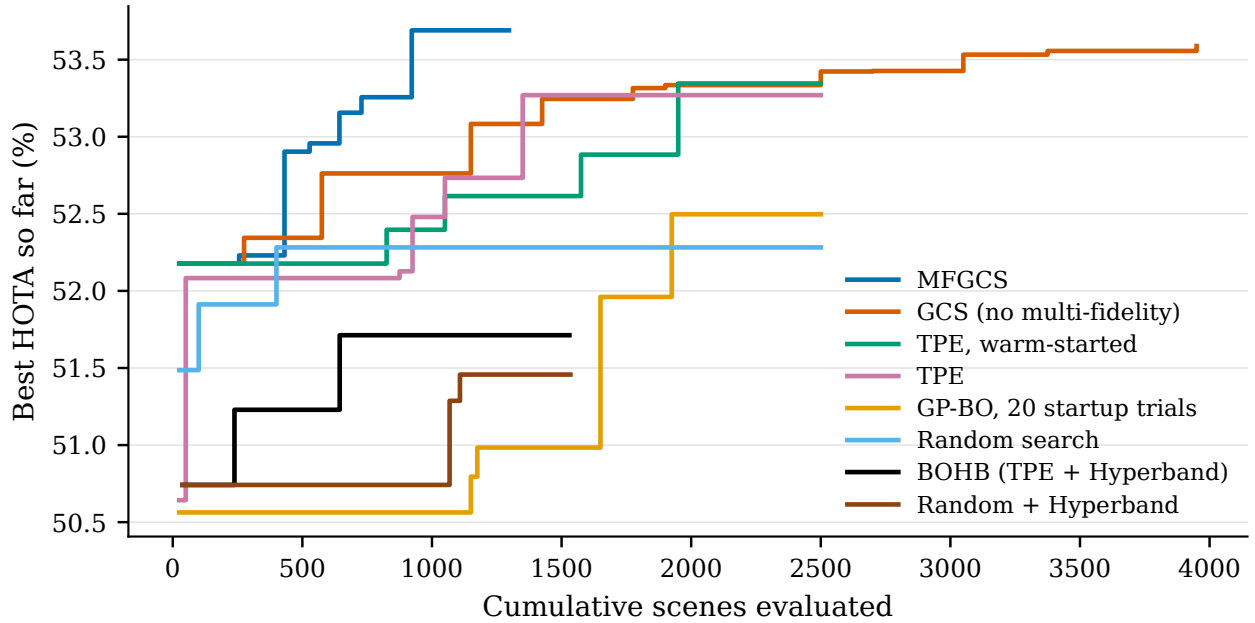


Figure 2: Running-best HOTA against cumulative scenes evaluated, on the SORT and DanceTrack-val search space, for the best configuration of each optimization method.

from a published 47.90 to 54.05 [2]. The search includes detection-selection thresholds as well as association parameters, so both terms are within its reach.

The same improvement is visible on the validation splits. Table 4 shows that both optimization methods outperform every published score and every configuration we tuned by hand. MFGCS reaches the per-tracker target faster than TPE on seven of the eight combinations.<sup>3</sup> The gains are not specific to one setting: both methods outperform the manual baseline in all eight validation settings, and on DanceTrack the validation-selected configuration moves by at most 0.58 HOTA points when it is evaluated on the test split. On SportsMOT all four score lower on the test split than on the validation split, by 6.06 to 8.83 points, with the largest drop on SparseTrack.

<sup>3</sup>The targets are fixed before the runs and shared by both optimizers within a cell, each a round HOTA value that both of them clear: 53.0 for SORT and ByteTrack, 57.0 for SparseTrack and 54.0 for MoveSORT-KF on DanceTrack; 78.0 for SORT, ByteTrack and SparseTrack and 79.0 for MoveSORT-KF on SportsMOT.

Table 3: Test-split results for all eight tuned configurations, each tracker paired with the better of TPE and MFGCS on its validation split. Our metrics are returned by the official evaluation servers. An em dash marks a tracker with no published test-split number. Published values are taken from Sun et al. [32] for SORT and ByteTrack on DanceTrack, Liu et al. [24] for SparseTrack, Adžemović et al. [2] for MoveSORT-KF, and Cui et al. [12] for ByteTrack on SportsMOT.

| Tracker                | Method | Tuned, this work |       |       |       |       |      | Published |       |
|------------------------|--------|------------------|-------|-------|-------|-------|------|-----------|-------|
|                        |        | HOTA             | DetA  | AssA  | MOTA  | IDF1  | IDSW | HOTA      | DetA  |
| <i>DanceTrack-test</i> |        |                  |       |       |       |       |      |           |       |
| SORT                   | MFGCS  | 54.05            | 79.41 | 36.89 | 90.50 | 55.14 | 1728 | 47.90     | 72.00 |
| ByteTrack              | TPE    | 53.73            | 80.71 | 35.88 | 92.13 | 54.70 | 1964 | 47.70     | 71.00 |
| SparseTrack            | MFGCS  | 58.17            | 81.94 | 41.44 | 91.42 | 56.68 | 1646 | 55.50     | 78.90 |
| MoveSORT-KF            | TPE    | 55.58            | 81.30 | 38.13 | 91.78 | 54.89 | 1716 | 53.30     | 80.70 |
| <i>SportsMOT-test</i>  |        |                  |       |       |       |       |      |           |       |
| SORT                   | TPE    | 72.38            | 87.74 | 59.78 | 96.32 | 72.08 | 2724 | —         | —     |
| ByteTrack              | MFGCS  | 72.79            | 87.94 | 60.32 | 96.57 | 72.68 | 2621 | 64.10     | 78.50 |
| SparseTrack            | TPE    | 69.52            | 87.37 | 55.39 | 96.02 | 68.46 | 3591 | —         | —     |
| MoveSORT-KF            | MFGCS  | 72.89            | 88.08 | 60.38 | 96.54 | 72.57 | 2617 | 71.00     | 86.20 |

Table 4: Validation results. *Paper* is the HOTA published in each tracker’s original work and *Manual* a configuration set by hand. The better of the two optimizers is in bold within each pair of columns. An em dash marks a cell with no published number. Published values have the same sources as in Table 3.

| Tracker               | HOTA  |        |              |              | Time-to-target |                |
|-----------------------|-------|--------|--------------|--------------|----------------|----------------|
|                       | Paper | Manual | TPE          | MFGCS        | TPE            | MFGCS          |
| <i>DanceTrack-val</i> |       |        |              |              |                |                |
| SORT                  | 47.80 | 52.18  | 53.27        | <b>53.69</b> | 125 min        | <b>73 min</b>  |
| ByteTrack             | 47.10 | 53.05  | <b>53.64</b> | 53.29        | 78 min         | <b>2.5 min</b> |
| SparseTrack           | 53.90 | 54.37  | 57.34        | <b>58.75</b> | 207 min        | <b>78 min</b>  |
| MoveSORT-KF           | 53.30 | 53.35  | <b>55.16</b> | 54.75        | 97 min         | <b>19 min</b>  |
| <i>SportsMOT-val</i>  |       |        |              |              |                |                |
| SORT                  | —     | 74.91  | <b>79.14</b> | 78.36        | 52 min         | <b>51 min</b>  |
| ByteTrack             | 62.80 | 75.88  | 78.85        | 78.85        | <b>30 min</b>  | 110 min        |
| SparseTrack           | —     | 75.56  | <b>78.35</b> | 78.20        | 195 min        | <b>118 min</b> |
| MoveSORT-KF           | —     | 76.99  | 79.67        | <b>80.19</b> | 77 min         | <b>48 min</b>  |

#### 5.4. Ablation studies

We ablate the scene sampler and the coordinate optimizer of Section 4.2 on the SORT and DanceTrack-val search space, at the same budget and seed as above. Table 5 reports the sweep and Figure C.3 shows the corresponding running-best curves. The per-coordinate early-stopping rule is held fixed across every run reported here, so we do not ablate it.

**Coordinate optimizer.** We compare three ways of proposing candidate values for one hyperparameter. Grid search evaluates  $g$  evenly spaced points across the current interval, including both endpoints, and contracts around the best of them. Ternary section instead evaluates two interior points per round and discards the outer third on the losing side, which halves the evaluations per round but can never return a value on a boundary. The random variant draws a fixed number of candidate values uniformly from the interval, with no contraction at all, and serves as the unstruc-

Table 5: MFGCS configuration ablation on the SORT and DanceTrack-val search space. The suffix  $\mathbf{g}\{\mathbf{N}\}\mathbf{r}\{\mathbf{R}\}\mathbf{-n}\{\mathbf{S}\}\mathbf{-s}\{\mathbf{T}\}$  denotes grid points per coordinate, contraction rounds, subset scenes, and maximum sweeps;  $\mathbf{tern}\{\mathbf{N}\}$  is the ternary variant and  $\mathbf{rand}\{\mathbf{M}\}$  the random variant with  $M$  candidates per coordinate. Grouped by coordinate optimizer, by descending HOTA within each group; the selected configuration is in bold.

| Configuration     | HOTA         | Scenes      | Observation                               |
|-------------------|--------------|-------------|-------------------------------------------|
| <i>Grid</i>       |              |             |                                           |
| <b>g5r3-n6-s6</b> | <b>53.69</b> | <b>1297</b> | <b>selected</b>                           |
| g5r3-n9-s6        | 53.33        | 1280        | subset noise already below the win margin |
| g7r2-n6-s6        | 53.32        | 1321        | wider lattice does not beat deeper rounds |
| g5r3-n3-s6        | 52.95        | 874         | subset too small, noisy comparisons       |
| g7r3-n6-s6        | 52.74        | 1293        | resolution past $g = 5$ over-shrinks      |
| g5r2-n6-s6        | 52.55        | 911         | two rounds are not enough                 |
| g4r2-n3-s6        | 52.18        | 254         | grid too coarse to escape the start       |
| <i>Ternary</i>    |              |             |                                           |
| tern5-n3-s6       | 53.19        | 997         | best ternary setting                      |
| tern5-n6-s6       | 52.40        | 1198        | larger subset hurts the ternary variant   |
| <i>Random</i>     |              |             |                                           |
| rand20-n6-s6      | 52.90        | 1553        | random improves at high candidate count   |
| rand5-n3-s6       | 52.18        | 221         | too few random candidates                 |

tured control. Appendix Appendix C.2 derives the evaluation-cost and contraction-rate trade-off between grid and ternary.

Grid search at **g5r3-n6-s6** ( $g = 5$  points per round,  $r = 3$  contraction rounds,  $m = 6$  subset scenes,  $T = 6$  sweeps) reaches 53.69% HOTA, ahead of the ternary variant at 53.19% and the random variant at 52.90%. We carry grid forward because it scored highest among the variants we tested here, and because its candidate set includes the interval endpoints, where several of these hyperparameters take their best observed values. Within grid search, the coarsest variant we tested did not improve on its starting configuration, because its lattice re-quantized the best point onto the value the search began from, and three contraction rounds beat two by 1.1 HOTA points, whereas resolution past  $g = 5$  does not help.

**Scene sampler.** The subset size trades comparison noise against budget, and the best observed setting depends on the coordinate optimizer. Among the sizes we tested, grid does best at  $m = 6$ , which reaches 53.69%: dropping to  $m = 3$  costs 0.74 points (52.95%) because the noisier comparison discards good candidates before they reach a full evaluation, while raising it to  $m = 9$  gives back 53.33% for 1280 scenes, buying less variance reduction than it costs in budget. For ternary the best observed setting is  $m = 3$  at 53.19%, against 52.40% at  $m = 6$ .

Setting  $m$  to all 25 scenes removes the multi-fidelity stage altogether, since every candidate is then scored on the full split. That run reaches 53.59% HOTA and consumes 3950 cumulative scenes, against MFGCS’s 53.69% at 1297; the 3950 scenes are 158 full-split equivalents, despite the run recording only 12 trials. Comparing the two end points understates the subset stage, because it allows the full-fidelity variant three times the budget. Read at matched cost, the full-fidelity variant stands at 53.08% after 1297 scenes, so the subset stage is worth 0.61 HOTA points at equal budget as well as cutting that budget threefold.

## 6. Conclusion

We benchmarked hyperparameter optimization for tracking-by-detection multi-object tracking, comparing eight optimization methods across six families. Within each tracker and dataset pair they share a search space and an evaluation pipeline, so the comparison isolates the search strategy. We introduced Multi-Fidelity Greedy Coordinate Search, an optimizer matched to the cost structure of the problem. Our main observations are as follows:

- The configurations these trackers are commonly run with are under-tuned on both benchmarks. On the validation splits both optimization methods outperform every published score and every configuration we tuned by hand, on all four trackers. Taking both optimizers across all cells, the margin over the published scores runs from 1.45 to 16.05 HOTA points, and over our own hand-tuned configurations from 0.24 to 4.38 points. For each setting the better validation-selected configuration also outperforms the published number on the held-out test split, by 1.9 to 8.7 points.
- Hand-tuning consumed a few hours to a full day per tracker and scored below both optimizers on every row, while MFGCS reached a higher-scoring configuration in under two hours of unattended wall-clock.
- MFGCS and TPE reach comparable HOTA in these runs, differing by at most 1.41 points in any cell and by under 0.8 in seven of the eight, and MFGCS reaches the per-tracker target faster on seven of the eight, and its scene-subset stage alone cuts the budget threefold while scoring 0.61 HOTA points above the full-fidelity coordinate search at equal cost. The reduction has to be paired with the search to pay off. Hyperband and BOHB reduce fidelity across independently sampled configurations instead, each scored on its own scene sample, and pruning on that unpaired signal cost 0.82 and 1.56 HOTA points against the same samplers without it.

Our results show that a tracker’s reported number and its achievable number are different quantities, and that the difference is large enough to change how two published methods compare. We release the optimized configurations so that a tuned baseline is available to compare against.

## Appendix A. Experimental details

This section provides the configurations summarized in Section 5.1. We first describe the object detectors, then the inference and postprocessing pipeline, then the search spaces, and finally the settings of each optimizer.

### *Appendix A.1. Object detection*

Both datasets use a YOLOX-X detector [16]. For DanceTrack we use the YOLOX-X checkpoint released with the ByteTrack ablation setup, and for SportsMOT the YOLOX-X checkpoint released with the dataset. Because the detector is frozen, detections are computed once per dataset and written to a cache addressed by scene name and frame index, so every trial reads identical detections and the objective stays deterministic. A warm-cache pass over DanceTrack-val costs two to three minutes, against roughly 1.5 hours with the detector in the loop.

### Appendix A.2. Tracker inference postprocessing

Tracker outputs are postprocessed before scoring using trajectory interpolation methods, with settings that differ per dataset. Gaps are filled by linear interpolation when the track is lost for at most 10 frames on DanceTrack, or 20 on SportsMOT, and the surrounding tracklet is at least 30 frames long. Final tracks shorter than 20 frames on DanceTrack, or 10 on SportsMOT, are removed. Every score reported in this paper, including every optimizer trial, is computed on the postprocessed output.

### Appendix A.3. Search spaces

Table A.6 lists every hyperparameter that is part of the tracker’s search space. The shared block applies to all four trackers; the remaining blocks are added per tracker. This gives 7 hyperparameters for SORT, 9 for MoveSORT-KF, 12 for ByteTrack, and 14 for SparseTrack, so the spaces differ in size by a factor of two across the four methods.

Table A.6: Search spaces. Ranges are inclusive.  $\dagger$  marks a hyperparameter whose candidates are generated at or above the sampled detection threshold. This constrains how candidates are proposed and is not an invariant of every stored configuration: a coordinate search that accepts no move for this parameter retains its existing value.  $\ddagger$  marks a logarithmic scale.

| Tracker     | Hyperparameter                          | Type        | Range                |
|-------------|-----------------------------------------|-------------|----------------------|
| All         | detection threshold                     | float       | [0.1, 0.9]           |
|             | initialization threshold                | int         | [0, 3]               |
|             | remember threshold                      | int         | [1, 60]              |
|             | motion-model position noise scale       | float       | [0.01, 0.25]         |
| SORT        | new-track detection threshold $\dagger$ | float       | [0.3, 0.9]           |
|             | IoU match threshold                     | float       | [0.10, 0.50]         |
|             | score fusion                            | categorical | {on, off}            |
| ByteTrack   | new-track detection threshold $\dagger$ | float       | [0.1, 0.9]           |
|             | duplicate IoU threshold                 | float       | [0.5, 1.0]           |
|             | high-tier match threshold               | float       | [0.10, 0.50]         |
|             | high-tier score fusion                  | categorical | {on, off}            |
|             | low-tier match threshold                | float       | [0.30, 0.70]         |
|             | low-tier score fusion                   | categorical | {on, off}            |
|             | new-track match threshold               | float       | [0.10, 0.50]         |
|             | new-track score fusion                  | categorical | {on, off}            |
| SparseTrack | new-track detection threshold $\dagger$ | float       | [0.1, 0.9]           |
|             | duplicate IoU threshold                 | float       | [0.5, 1.0]           |
|             | high-tier match threshold               | float       | [0.10, 0.50]         |
|             | high-tier score fusion                  | categorical | {on, off}            |
|             | high-tier depth levels                  | int         | [6, 20]              |
|             | low-tier match threshold                | float       | [0.30, 0.70]         |
|             | low-tier score fusion                   | categorical | {on, off}            |
|             | low-tier depth levels                   | int         | [6, 20]              |
|             | new-track match threshold               | float       | [0.10, 0.50]         |
|             | new-track score fusion                  | categorical | {on, off}            |
| MoveSORT-KF | new-track detection threshold $\dagger$ | float       | [0.1, 0.9]           |
|             | Move match threshold                    | float       | [0.10, 0.50]         |
|             | score fusion                            | categorical | {on, off}            |
|             | motion weight $\lambda^\ddagger$        | float       | [1.0, 15.0]          |
|             | motion distance                         | categorical | $\{\ell_1, \ell_2\}$ |

#### Appendix A.4. Optimizer configurations

Every optimizer except MFGCS is used from the Optuna framework [5]. The TPE sampler is Optuna’s `TPESampler` with a univariate kernel, 24 expected-improvement candidates per step, and a good-or-bad split quantile of  $\gamma = 0.20$ . The quantile is a setting we selected, not an Optuna default; Optuna’s own default is  $\min(\lceil 0.1n \rceil, 25)$ . Optuna’s `GPSampler` uses expected improvement with 20 startup trials in the configuration reported in Table 2, and 10 in the default variant, and the Hyperband variants use a rung schedule over scene subsets with a reduction factor of 3. Random search samples uniformly over the same space. Every optimizer uses seed 42 and a cap of 100 full-fidelity evaluations. Table A.7 gives the MFGCS settings.

Table A.7: MFGCS default configuration, abbreviated **g5r3-n6-s6** in the main text.

| Parameter            | Default        | Purpose                                                                     |
|----------------------|----------------|-----------------------------------------------------------------------------|
| coordinate optimizer | grid           | Coarse-to-fine grid search per coordinate.                                  |
| $g$ , grid points    | 5              | Points per round, including both endpoints.                                 |
| $r$ , rounds         | 3              | Contraction rounds per coordinate.                                          |
| scene sampler        | uniform random | Draws the low-fidelity subset.                                              |
| $m$ , subset size    | 6              | Scenes drawn per coordinate at low fidelity.                                |
| $T$ , max sweeps     | 6              | Outer-loop cap.                                                             |
| max trials           | 100            | Hard cap on full-fidelity evaluations.                                      |
| early stop           | on             | Stop after a sweep with no accepted move.                                   |
| $\varepsilon$        | $10^{-4}$      | Minimum HOTA gain to accept, at the noise floor $\sigma_d/\sqrt{m}$ .       |
| barren-sweep drop    | 2              | Sweeps without an accept before a coordinate is dropped.                    |
| $\rho$ , shrink      | 0.25           | Half-width of the post-accept interval, as a fraction of the current width. |
| bootstrap            | on             | One full evaluation of the initial configuration seeds $v$ .                |

## Appendix B. Additional results

This section reports the full metric panel behind Table 4, the optimized hyperparameter values themselves, and the information needed to reproduce them.

#### Appendix B.1. Full metrics and optimized hyperparameters

Table B.8 reports every metric at the best-HOTA trial each optimizer selected. The winner on HOTA is not always the winner on identity switches: on DanceTrack MoveSORT-KF, TPE leads on HOTA, DetA, AssA, MOTA and IDF1 while MFGCS commits 98 fewer identity switches.

Table B.9 and Table B.10 give the stored configuration values each optimizer selected, i.e. the values that produce the HOTA scores of Table 4. For SORT and MoveSORT-KF the tracker first filters detections by the detection threshold and only then applies the new-track gate, so the

Table B.8: Validation metrics at the best-HOTA trial; better of the two optimizers in bold.

| Tracker               | Method | HOTA         | DetA         | AssA         | MOTA         | IDF1         | IDSW        | FPS       |
|-----------------------|--------|--------------|--------------|--------------|--------------|--------------|-------------|-----------|
| <i>DanceTrack-val</i> |        |              |              |              |              |              |             |           |
| SORT                  | TPE    | 53.27        | <b>78.33</b> | 36.36        | <b>89.86</b> | 53.26        | 1669        | 930–1082  |
|                       | MFGCS  | <b>53.69</b> | 76.47        | <b>37.81</b> | 87.87        | <b>55.60</b> | <b>1573</b> |           |
| ByteTrack             | TPE    | <b>53.64</b> | 78.60        | <b>36.77</b> | 90.52        | <b>54.14</b> | 1842        | 1012–1121 |
|                       | MFGCS  | 53.29        | <b>79.08</b> | 36.04        | <b>90.66</b> | 53.68        | <b>1652</b> |           |
| SparseTrack           | TPE    | 57.34        | 77.32        | 42.65        | 87.12        | 58.52        | 1584        | 698–755   |
|                       | MFGCS  | <b>58.75</b> | <b>78.86</b> | <b>43.95</b> | <b>89.67</b> | <b>58.73</b> | <b>1524</b> |           |
| MoveSORT-KF           | TPE    | <b>55.16</b> | <b>78.61</b> | <b>38.87</b> | <b>90.27</b> | <b>54.96</b> | 1559        | 845–964   |
|                       | MFGCS  | 54.75        | 78.06        | 38.53        | 88.72        | 54.50        | <b>1461</b> |           |
| <i>SportsMOT-val</i>  |        |              |              |              |              |              |             |           |
| SORT                  | TPE    | <b>79.14</b> | 92.14        | <b>67.99</b> | 98.73        | <b>79.14</b> | <b>625</b>  | 930–1082  |
|                       | MFGCS  | 78.36        | <b>92.38</b> | 66.49        | <b>98.80</b> | 77.93        | 660         |           |
| ByteTrack             | TPE    | 78.85        | <b>92.36</b> | 67.34        | 98.81        | 78.43        | 629         | 1012–1121 |
|                       | MFGCS  | 78.85        | 92.29        | <b>67.39</b> | <b>98.83</b> | <b>78.78</b> | <b>625</b>  |           |
| SparseTrack           | TPE    | <b>78.35</b> | <b>92.11</b> | <b>66.66</b> | 98.45        | <b>78.32</b> | <b>731</b>  | 698–755   |
|                       | MFGCS  | 78.20        | 91.98        | 66.51        | <b>98.53</b> | 78.03        | 827         |           |
| MoveSORT-KF           | TPE    | 79.67        | 92.31        | 68.78        | 98.64        | 79.56        | 595         | 845–964   |
|                       | MFGCS  | <b>80.19</b> | <b>92.48</b> | <b>69.55</b> | <b>98.83</b> | <b>80.27</b> | <b>552</b>  |           |

effective confidence required to start a track is the larger of the two. The DanceTrack MoveSORT-KF column stores 0.75 and 0.70, for instance, giving an effective threshold of 0.75; the stored 0.70 gate admits nothing further. The searches selected substantially different values between datasets for the same tracker.

## Appendix C. Additional ablations

This section expands the ablation findings of Section 5.4: the MFGCS configuration sweep, the configurations of the baseline optimizers, and the formal comparison of the two contractive coordinate optimizers.

### Appendix C.1. MFGCS configuration

Table 5 reports the sweep over the MFGCS coordinate-search and subset-sampling parameters, and Figure C.3 shows the corresponding running-best curves. Grid search gives the best configuration overall, at 53.69% against 53.19% for the best ternary variant and 52.90% for the best random one, which needs 20 candidates per coordinate and 1553 scenes to get there. The setting that matters most is the grid resolution  $g$ , because too coarse a lattice re-quantizes the optimum onto the point the search began from and the sweep never moves: **g4r2-n3-s6** ties the untuned baseline at 52.18% after only 254 scenes. Given  $g = 5$ , three contraction rounds beat two by over a point (53.69% against 52.55%) at a comparable budget, while  $g = 7$  does not help at either depth, and the subset size trades noise against budget in both directions:  $m = 3$  discards good candidates before they reach a full evaluation (52.95%), whereas  $m = 9$  spends 1280 scenes to land within noise of  $m = 6$ .

Table B.9: Optimized SORT and MoveSORT-KF hyperparameters. The MoveSORT-KF SportsMOT MFGCS column is the configuration evaluated on SportsMOT-test in Table 3.

| Hyperparameter                | DanceTrack |          | SportsMOT |          |
|-------------------------------|------------|----------|-----------|----------|
|                               | TPE        | MFGCS    | TPE       | MFGCS    |
| <i>SORT</i>                   |            |          |           |          |
| detection threshold           | 0.5498     | 0.2      | 0.186     | 0.6      |
| initialization threshold      | 3          | 3        | 1         | 3        |
| remember threshold            | 25         | 28       | 43        | 30       |
| position noise scale          | 0.0257     | 0.05     | 0.0144    | 0.025    |
| match threshold               | 0.2254     | 0.325    | 0.1288    | 0.1      |
| score fusion                  | off        | off      | on        | off      |
| new-track detection threshold | 0.8417     | 0.9      | 0.8147    | 0.6562   |
| HOTA                          | 53.27      | 53.69    | 79.14     | 78.36    |
| <i>MoveSORT-KF</i>            |            |          |           |          |
| detection threshold           | 0.648      | 0.75     | 0.7337    | 0.675    |
| initialization threshold      | 1          | 0        | 3         | 3        |
| remember threshold            | 25         | 30       | 49        | 30       |
| position noise scale          | 0.0373     | 0.05     | 0.0171    | 0.01     |
| match threshold               | 0.204      | 0.2      | 0.145     | 0.1      |
| score fusion                  | off        | off      | off       | off      |
| motion weight $\lambda$       | 7.7777     | 3.873    | 5.6847    | 7.622    |
| motion distance               | $\ell_2$   | $\ell_1$ | $\ell_1$  | $\ell_1$ |
| new-track detection threshold | 0.7454     | 0.7      | 0.7693    | 0.7      |
| HOTA                          | 55.16      | 54.75    | 79.67     | 80.19    |

### Appendix C.2. Ternary section and grid search comparison

The ternary and grid coordinate optimizers of Section 4.2 are both one-dimensional contractive search methods, but they are not equivalent. Ternary section is not a special case of grid search for any choice of  $g$  and  $r$ .

#### Appendix C.2.1. Evaluation points

For a window  $[A, B]$  of width  $w = B - A$ , ternary section evaluates the two interior points  $m_1 = A + w/3$  and  $m_2 = B - w/3$ , never the endpoints. Grid search with  $g$  points evaluates  $g$  evenly spaced points including both endpoints, so  $g = 4$  evaluates  $\{A, A + w/3, A + 2w/3, B\}$ , i.e. both ternary points *plus* the endpoints. No setting of  $g$  reproduces the ternary evaluation set, because grid always includes the endpoints and ternary never does.

#### Appendix C.2.2. Contraction rule

Ternary discards the third of the window on the far side of the better interior point, so the new width is always  $\frac{2}{3}w$  regardless of where the optimum lies. Grid recenters on the best point,  $[\hat{x} - \delta, \hat{x} + \delta]$  with  $\delta = w/(g - 1)$ , giving  $2w/(g - 1)$  for an interior optimum and  $w/(g - 1)$  when the optimum is at an endpoint and the window is clipped. At  $g = 4$  that is  $\frac{2}{3}w$  interior, the same rate as ternary, and  $\frac{1}{3}w$  at an endpoint, which is faster.

Table B.10: Optimized ByteTrack and SparseTrack hyperparameters. The SparseTrack DanceTrack MFGCS column is the configuration evaluated on DanceTrack-test in Table 3.

| Hyperparameter                | DanceTrack |        | SportsMOT |        |
|-------------------------------|------------|--------|-----------|--------|
|                               | TPE        | MFGCS  | TPE       | MFGCS  |
| <i>ByteTrack</i>              |            |        |           |        |
| detection threshold           | 0.5254     | 0.6    | 0.3443    | 0.55   |
| initialization threshold      | 3          | 3      | 0         | 3      |
| remember threshold            | 21         | 19     | 6         | 30     |
| position noise scale          | 0.0793     | 0.05   | 0.0158    | 0.0225 |
| duplicate IoU threshold       | 0.9996     | 1      | 0.977     | 1      |
| high-tier match threshold     | 0.1987     | 0.25   | 0.1001    | 0.1    |
| high-tier score fusion        | off        | off    | on        | off    |
| low-tier match threshold      | 0.313      | 0.5    | 0.6266    | 0.45   |
| low-tier score fusion         | on         | off    | on        | off    |
| new-track match threshold     | 0.4264     | 0.25   | 0.1043    | 0.1    |
| new-track score fusion        | on         | off    | off       | off    |
| new-track detection threshold | 0.6972     | 0.7    | 0.3818    | 0.6812 |
| HOTA                          | 53.64      | 53.29  | 78.85     | 78.85  |
| <i>SparseTrack</i>            |            |        |           |        |
| detection threshold           | 0.6439     | 0.6    | 0.7308    | 0.4    |
| initialization threshold      | 0          | 3      | 0         | 3      |
| remember threshold            | 24         | 14     | 14        | 10     |
| position noise scale          | 0.2387     | 0.2172 | 0.01      | 0.05   |
| duplicate IoU threshold       | 0.8567     | 0.9062 | 0.9987    | 1      |
| high-tier match threshold     | 0.2804     | 0.35   | 0.1235    | 0.1063 |
| high-tier score fusion        | off        | off    | off       | off    |
| high-tier depth levels        | 16         | 12     | 17        | 12     |
| low-tier match threshold      | 0.6909     | 0.65   | 0.473     | 0.7    |
| low-tier score fusion         | off        | off    | off       | off    |
| low-tier depth levels         | 20         | 16     | 7         | 12     |
| new-track match threshold     | 0.4122     | 0.25   | 0.2371    | 0.1063 |
| new-track score fusion        | on         | off    | on        | off    |
| new-track detection threshold | 0.8937     | 0.8062 | 0.7393    | 0.7    |
| HOTA                          | 57.34      | 58.75  | 78.35     | 78.20  |

### Appendix C.2.3. Evaluation cost

Halving the window costs ternary about 3.4 evaluations and grid at  $g = 4$  about 6.8 for an interior optimum, but only 2.5 at an endpoint. Ternary is therefore roughly twice as cheap for the same contraction rate, *provided* the optimum is interior and the objective is strictly unimodal; grid reaches parity or wins otherwise. On integer coordinates the two ternary points frequently round to the same lattice point over a short range, which makes the quantized grid lattice more reliable, and on unordered categoricals both degenerate to enumeration.

### Appendix C.3. Baseline optimizer configurations

**TPE.** We ran several experiments to tune TPE beyond the configuration we selected, and none of them improved on it by more than noise, which is why that configuration is the one carried into the cross-tracker experiment. The warm-started variant records the highest score of the TPE family

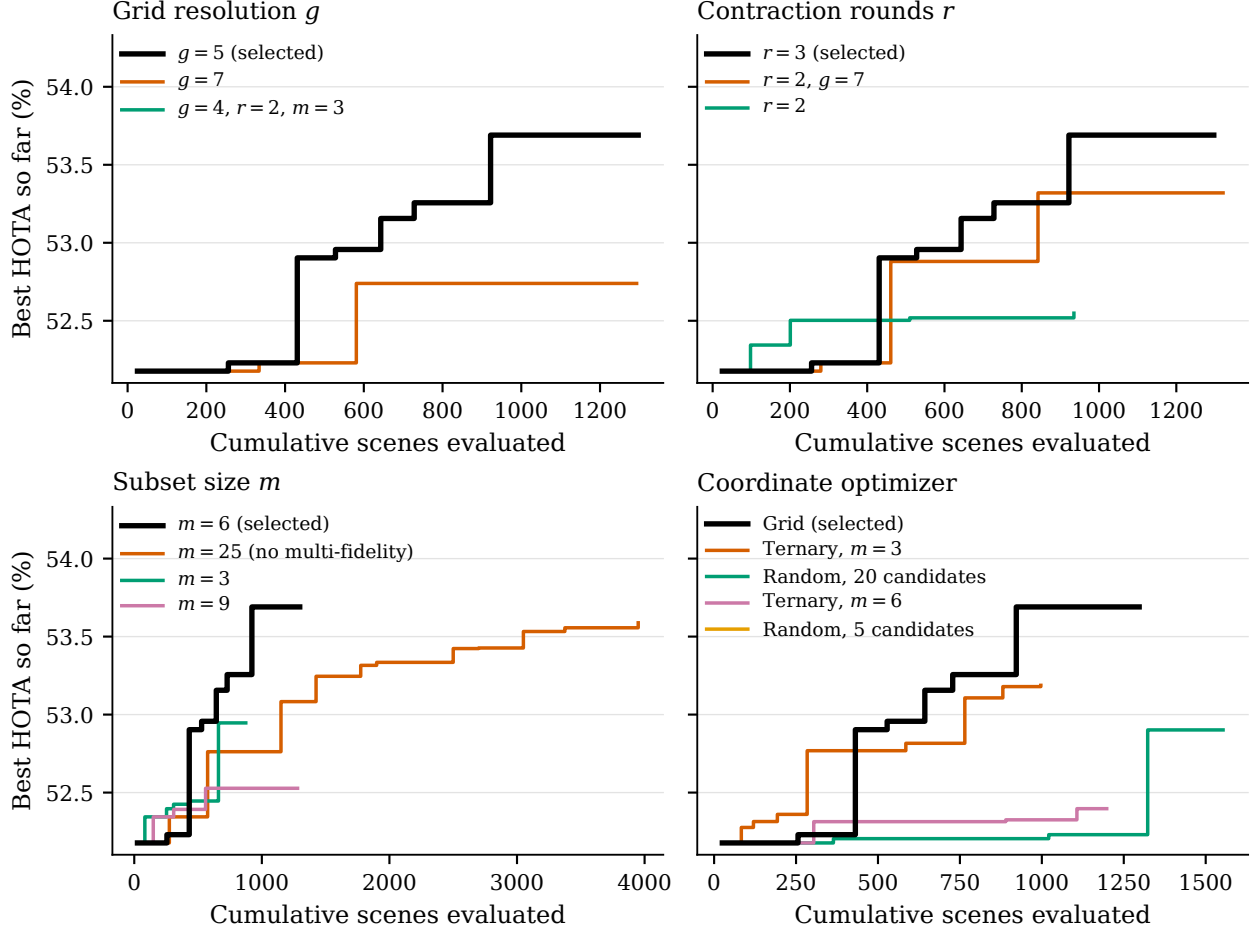


Figure C.3: MFGCS configuration ablation on the SORT and DanceTrack-val search space. Each panel varies one setting and repeats the selected configuration in black as the common reference. Note the differing horizontal scales: the  $m = 25$  run evaluates every scene at every step and so spends roughly three times the budget of the others.

and is the one plotted in Figure 2; plain TPE is the one selected, because warm-starting depends on the manually tuned configuration. Widening the candidate pool, switching to the multivariate kernel, and warm-starting from the manual configuration all land within a point of it, while the only setting that hurts materially is a split quantile of  $\gamma = 0.30$ , which loses 1.5 points because a larger quantile blurs the density ratio driving the acquisition. All variants in Table C.11 run 100 trials and consume 2500 cumulative scenes, and their reported durations are all approximately 258 minutes, so wall-clock is omitted. Figure C.4 shows the running-best curves.

**Gaussian process.** We likewise varied the GP sampler, and no setting lifted it clearly above the model-free baseline: two of the three variants in Table C.12 sit below the 52.28% of uniform random search and the third reaches 52.50%, a margin this comparison does not resolve. Doubling the startup trials scores slightly higher, and seeding the sampler with the manual configuration scores slightly lower. The three variants run 100 trials, consume 2500 cumulative scenes, and take about 259 minutes; Figure C.5 shows the running-best curves.

**Hyperband pruning.** Table C.13 pairs each Hyperband-pruned method with the same sampler run without pruning, which isolates pruning as a factor across two samplers. Pruning works as

Table C.11: TPE configuration ablation on the SORT and DanceTrack-val search space, sorted by HOTA. Every row is the TPE sampler; the *Sampler* column lists what differs from its defaults. The variant carried into Section 5.3 is in bold.

| Configuration            | Sampler                                                     | HOTA         |
|--------------------------|-------------------------------------------------------------|--------------|
| tpe_sort_gamma030        | $\gamma = 0.30$                                             | 51.77        |
| tpe_sort_neic64          | 24 $\rightarrow$ 64 EI candidates                           | 52.81        |
| tpe_sort_multivariate    | multivariate kernel                                         | 52.88        |
| tpe_sort_neic64_gamma010 | $\gamma = 0.10$ , 64 EI candidates                          | 53.24        |
| tpe_sort                 | <b>selected (<math>\gamma = 0.20</math>, 24 candidates)</b> | <b>53.27</b> |
| tpe-prior_sort           | warm-started with the manually-tuned config                 | 53.35        |

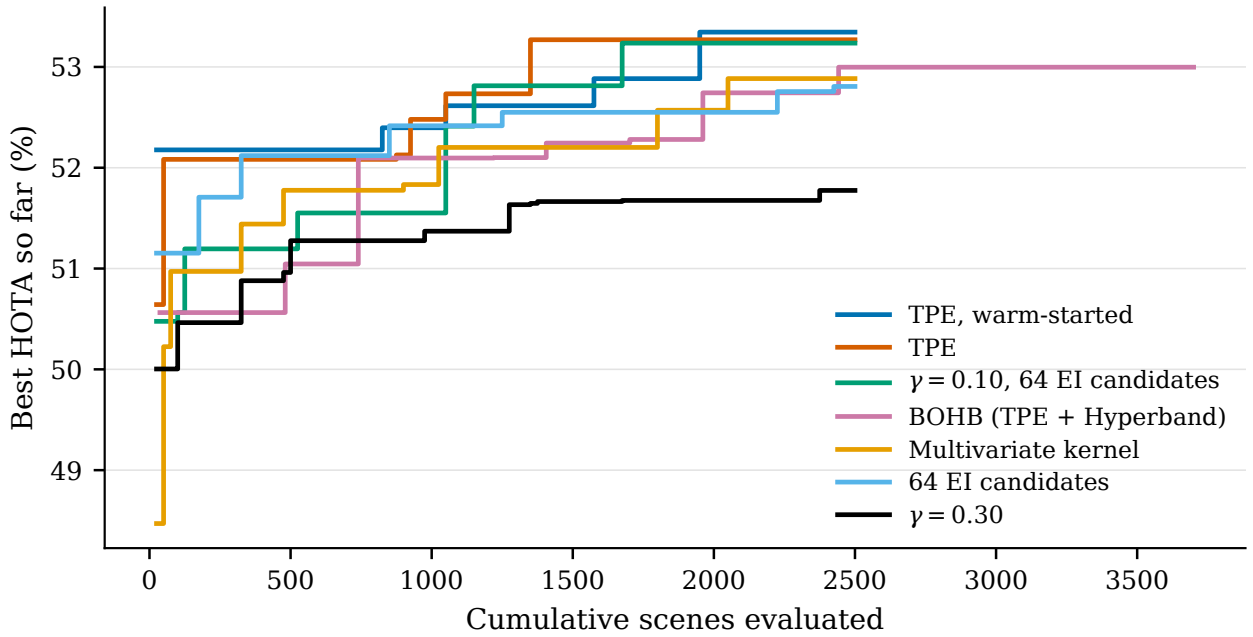


Figure C.4: Running-best HOTA per TPE variant against cumulative scenes evaluated.

Table C.12: Gaussian-process configuration ablation on the SORT and DanceTrack-val search space, sorted by HOTA.

| Configuration    | Sampler                                               | HOTA  |
|------------------|-------------------------------------------------------|-------|
| sort-gp-prior    | prior-seeded, manually-tuned configuration at trial 0 | 52.18 |
| sort-gp          | 10 startup trials                                     | 52.23 |
| sort-gp-warmup20 | 20 startup trials                                     | 52.50 |

designed on the budget axis: it removes 78% and 76% of trials and cuts the cost from 2500 scenes to 1534 and 1530. It also costs accuracy. Read at matched budget rather than at each run’s end point, the unpruned samplers have already reached 52.28% by 1534 scenes and 53.27% by 1530, so pruning costs 0.82 and 1.56 HOTA points at equal cost.

We attribute this to the structure of the low-fidelity comparison rather than to pruning as such.

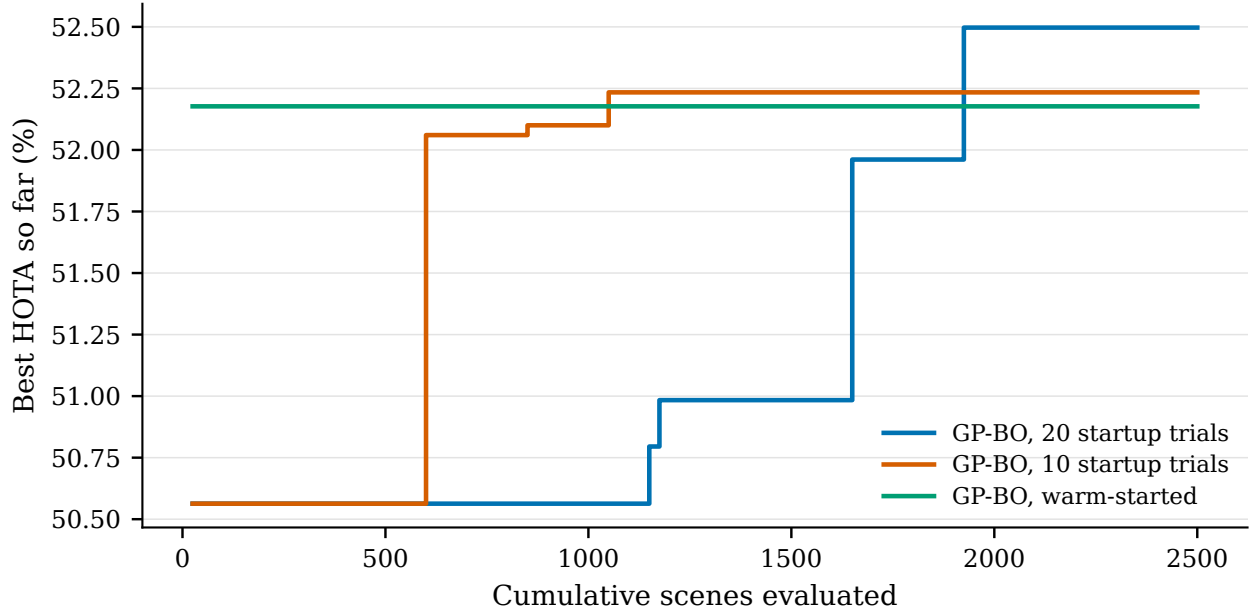


Figure C.5: Running-best HOTA per Gaussian-process variant against cumulative scenes evaluated.

Hyperband ranks whole configurations against one another at a rung, and each trial draws its own scene subset, so the sampled scenes and the configuration change are confounded and the noise does not cancel. MFGCS reduces fidelity on the same axis but compares candidate values of a single coordinate, with the others held fixed and all candidates scored on one shared sample, so the per-scene component is common to both sides of every comparison. This is an explanation of the observed difference, not a measurement of the mechanism, and the pairing is not intrinsic: fixing one subset per rung across trials would pair Hyperband’s comparisons too. Each cell is a single run at seed 42.

Table C.13: Effect of Hyperband pruning, each pruned method paired with the same sampler run without it, plus the coordinate-greedy run without the multi-fidelity stage. Study names omit their shared `sort-` prefix.

| Study                              | Algorithm                            | HOTA  | Trials | Scenes |
|------------------------------------|--------------------------------------|-------|--------|--------|
| <code>random</code>                | Random, no pruning                   | 52.28 | 100    | 2500   |
| <code>hyperband-random</code>      | Random + Hyperband                   | 51.46 | 100    | 1534   |
| <code>tpe</code>                   | TPE, no pruning                      | 53.27 | 100    | 2500   |
| <code>bohb</code>                  | TPE + Hyperband                      | 51.71 | 100    | 1530   |
| <code>gcs-no-mf-g5r3-n25-s6</code> | Coordinate greedy, no multi-fidelity | 53.59 | 12     | 3950   |

## References

- [1] M. Adžemović. Deep learning-based multi-object tracking: A comprehensive survey from foundations to state-of-the-art. *arXiv preprint*, 2025. arXiv:2506.13457.
- [2] M. Adžemović, P. Tadić, A. Petrović, and M. Nikolić. Beyond Kalman filters: deep learning-based filters for improved object tracking. *Machine Vision and Applications*, 36(1):20, 2024. doi: 10.1007/s00138-024-01644-x.
- [3] M. Adžemović, P. Tadić, A. Petrović, and M. Nikolić. Engineering an efficient object tracker for non-linear motion. *The Visual Computer*, 2025. doi: 10.1007/s00371-025-04243-7.
- [4] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky. BoT-SORT: Robust associations multi-pedestrian tracking. *arXiv preprint*, 2022. arXiv:2206.14651.
- [5] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama. Optuna: A next-generation hyperparameter optimization framework. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631, 2019. doi: 10.1145/3292500.3330701.
- [6] J. Bergstra and Y. Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.
- [7] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl. Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 24, pages 2546–2554, 2011.
- [8] K. Bernardin and R. Stiefelhagen. Evaluating multiple object tracking performance: the CLEAR MOT metrics. *EURASIP Journal on Image and Video Processing*, 2008:1–10, 2008. doi: 10.1155/2008/246309.
- [9] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft. Simple online and realtime tracking. In *IEEE International Conference on Image Processing (ICIP)*, pages 3464–3468, 2016. doi: 10.1109/ICIP.2016.7533003.
- [10] J. Cao, J. Pang, X. Weng, R. Khirodkar, and K. Kitani. Observation-centric SORT: Rethinking SORT for robust multi-object tracking. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9686–9696, 2023.
- [11] X. Cao, Y. Zheng, Y. Yao, H. Qin, X. Cao, and S. Guo. TOPIC: A parallel association paradigm for multi-object tracking under complex motions and diverse scenes. *IEEE Transactions on Image Processing*, 34:743–758, 2025. doi: 10.1109/TIP.2025.3526066.
- [12] Y. Cui, C. Zeng, X. Zhao, Y. Yang, G. Wu, and L. Wang. SportsMOT: A large multi-object tracking dataset in multiple sports scenes. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9921–9931, 2023.
- [13] S. Falkner, A. Klein, and F. Hutter. BOHB: Robust and efficient hyperparameter optimization at scale. In *International Conference on Machine Learning (ICML)*, pages 1437–1446, 2018.
- [14] M. Feurer and F. Hutter. Hyperparameter optimization. In *Automated Machine Learning: Methods, Systems, Challenges*, pages 3–33. Springer, 2019.

- [15] T. Fleck and J. M. Zöllner. Tuning multi object tracking systems using Bayesian optimization. In *IEEE 24th International Conference on Information Fusion (FUSION)*, pages 1–8, 2021.
- [16] Z. Ge, S. Liu, F. Wang, Z. Li, and J. Sun. YOLOX: Exceeding YOLO series in 2021. *arXiv preprint*, 2021. arXiv:2107.08430.
- [17] A. Geiger, P. Lenz, and R. Urtasun. Are we ready for autonomous driving? the KITTI vision benchmark suite. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3354–3361, 2012. doi: 10.1109/CVPR.2012.6248074.
- [18] F. Hutter, H. H. Hoos, and K. Leyton-Brown. Sequential model-based optimization for general algorithm configuration. In *Learning and Intelligent Optimization (LION)*, pages 507–523, 2011.
- [19] M. M. Itoh, Q. Hu, T. Niizato, H. Kawashima, and K. Fujii. Fish tracking challenge 2024: A multi-object tracking competition with sweetfish schooling data. *arXiv preprint*, 2024. arXiv:2409.00339.
- [20] K. Jamieson and A. Talwalkar. Non-stochastic best arm identification and hyperparameter optimization. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 240–248, 2016.
- [21] T. G. Kolda, R. M. Lewis, and V. Torczon. Optimization by direct search: New perspectives on some classical and modern methods. *SIAM Review*, 45(3):385–482, 2003. doi: 10.1137/S003614450242889.
- [22] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52, 2018.
- [23] L. Li, K. Jamieson, A. Rostamizadeh, E. Gonina, J. Ben-Tzur, M. Hardt, B. Recht, and A. Talwalkar. A system for massively parallel hyperparameter tuning. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 2, pages 230–246, 2020.
- [24] Z. Liu, X. Wang, C. Wang, W. Liu, and X. Bai. SparseTrack: Multi-object tracking by performing scene decomposition based on pseudo-depth. *arXiv preprint*, 2023. arXiv:2306.05238.
- [25] J. Luiten, A. Ošep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé, and B. Leibe. HOTA: A higher order metric for evaluating multi-object tracking. *International Journal of Computer Vision*, 129(2):548–578, 2021. doi: 10.1007/s11263-020-01375-2.
- [26] M. Lurbur, K. C. Wilson, and N. Yochum. Towards automated bycatch monitoring: Optimizing and evaluating multi-object tracking of salmon in pollock trawls. *Ecological Informatics*, 94: 103674, 2026. doi: 10.1016/j.ecoinf.2026.103674.
- [27] F. Madrigal, C. Maurice, and F. Lerasle. Hyper-parameter optimization tools comparison for multiple object tracking applications. *Machine Vision and Applications*, 30(2):269–289, 2019. doi: 10.1007/s00138-018-0984-1.

- [28] I. S. Mohamed and K. C. Lim. PAE: Portable appearance extension for multiple object detection and tracking in traffic scenes. *IEEE Access*, 10:37257–37268, 2022. doi: 10.1109/ACCESS.2022.3160424.
- [29] E. Ristani, F. Solera, R. Zou, R. Cucchiara, and C. Tomasi. Performance measures and a data set for multi-target, multi-camera tracking. In *European Conference on Computer Vision (ECCV) Workshops*, pages 17–35, 2016.
- [30] J. Snoek, H. Larochelle, and R. P. Adams. Practical Bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 25, pages 2951–2959, 2012.
- [31] T. Stanczyk, S. Yoon, and F. Bremond. No train yet gain: Towards generic multi-object tracking in sports and beyond. *arXiv preprint*, 2025. arXiv:2506.01373.
- [32] P. Sun, J. Cao, Y. Jiang, Z. Yuan, S. Bai, K. Kitani, and P. Luo. DanceTrack: Multi-object tracking in uniform appearance and diverse motion. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 20993–21002, 2022.
- [33] O. Urbann, O. Bredtmann, M. Otten, J.-P. Richter, T. Bauer, and D. Zibriczky. Online and real-time tracking in a surveillance scenario. In *5th Workshop on Long-term Human Motion Prediction*, 2021. arXiv:2106.01153.
- [34] N. Wojke, A. Bewley, and D. Paulus. Simple online and realtime tracking with a deep association metric. In *IEEE International Conference on Image Processing (ICIP)*, pages 3645–3649, 2017. doi: 10.1109/ICIP.2017.8296962.
- [35] S. J. Wright. Coordinate descent algorithms. *Mathematical Programming*, 151(1):3–34, 2015. doi: 10.1007/s10107-015-0892-3.
- [36] Z. Xu, X. Zhan, Y. Xiu, C. Suzuki, and K. Shimada. Onboard dynamic-object detection and tracking for autonomous robot navigation with RGB-D camera. *IEEE Robotics and Automation Letters*, 9(1):651–658, 2024. doi: 10.1109/LRA.2023.3334683.
- [37] M. Yang, G. Han, B. Yan, W. Zhang, J. Qi, H. Lu, and D. Wang. Hybrid-SORT: Weak cues matter for online multi-object tracking. In *AAAI Conference on Artificial Intelligence*, pages 6504–6512, 2024.
- [38] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang. ByteTrack: Multi-object tracking by associating every detection box. In *European Conference on Computer Vision (ECCV)*, pages 1–21, 2022.